

11 · 工程实现方案

2026-07-31

Contents

11 · 工程实现方案

1. 技术栈选型

1.1 选型理由

2. 模块间接口

2.1 接口契约原则

3. 项目结构

4. 端到端编排 (Pipeline)

5. 部署方案

5.1 单机部署 (Docker Compose)

5.2 模型部署

5.3 扩展性

6. 可观测性

6.1 结构化日志

6.2 verification_log

6.3 指标监控

6.4 回放

7. 测试策略

7.1 回归集

7.2 评估集

8. 配置管理

9. 设计路线小结

1

1

2

3

4

5

6

6

6

6

6

6

6

6

6

6

7

7

7

7

11 · 工程实现方案

本文档阐述系统的技术栈选型、模块间接口定义、项目结构、部署方案、可观测性与测试策略。

1. 技术栈选型

层	技术	说明
语言	Python 3.11	主语言
视觉-传统	OpenCV	图像处理、Hough、轮廓、拟合
视觉-分割	SAM (segment-anything)	零样本分割
视觉-检测	YOLOv8 (ultralytics)	标注符号检测

层	技术	说明
视觉-OCR	PaddleOCR	标签文字识别
图结构	NetworkX	Geometry Graph 存储/查询
数据校验	Pydantic v2	Schema 约束
数学-符号	SymPy	代数/方程/三角
数学-SMT	Z3 (pyz3)	约束可满足性
数学-形式化	Lean 4 (可选)	证明校验
LLM	GPT-4o / Claude / Qwen2.5	推理 Agent
LLM 编排	LangChain / 自研	Agent 调度
向量库	FAISS / Chroma	定理 RAG
DSL 解析	lark	EBNF 解析
接口	FastAPI	模块间/对外 API
配置	YAML + Pydantic Settings	DSL 与配置
任务队列	Celery / RQ (可选)	异步解题
日志	structlog + JSON Lines	结构化日志
测试	pytest + 合成回归集	单测 + 回归
容器	Docker / Docker Compose	部署

1.1 选型理由

- **Python** 生态覆盖 CV/数学/LLM，团队熟悉。
- **开源模型优先** (SAM/YOLO/PaddleOCR/Qwen)，契合”不依赖大规模后训练”。
- **Pydantic** 保证数据契约类型安全。
- **NetworkX** 图操作灵活，支持多重图。

2. 模块间接口

所有模块以 HTTP/gRPC + JSON Schema 通信，关键接口用 Pydantic 定义：

```
# ===== Perception =====
class ParseRequest(BaseModel):
    image: bytes
    text: str

class PrimitiveSet(BaseModel):
    points: List[Point]
    segments: List[Segment]
    circles: List[Circle]
    ellipses: List[Ellipse]
    arcs: List[Arc]
    polygons: List[Polygon]
    marks: List[Mark]
    metadata: Metadata

def parse(req: ParseRequest) -> PrimitiveSet: ...

# ===== Graph =====
```

```

class GeometryGraph(BaseModel):
    nodes: List[Node]
    edges: List[Edge]
    metadata: Metadata

def build_graph(primitives: PrimitiveSet) -> GeometryGraph: ...

# ===== Relation Agents =====
class RelationCandidate(BaseModel):
    src: str; dst: str; rel: str
    evidence: str; confidence: float; attrs: dict = {}

def extract_relations(graph: GeometryGraph) -> List[RelationCandidate]: ...

# ===== Verifier =====
class VerifyResult(BaseModel):
    verified: Literal["true", "false", "uncertain"]
    evidence: str; measured: dict = {}; attrs: dict = {}

def verify(candidates: List[RelationCandidate], graph: GeometryGraph) -> GeometryGraph: ...

# ===== DSL =====
def to_dsl(graph: GeometryGraph) -> str: ...
def from_dsl(dsl: str) -> GeometryGraph: ...

# ===== LLM Reasoning =====
class ProofPlan(BaseModel):
    plan: List[Step]
    tools_calls: List[ToolCall]

def reason(dsl: str, problem: str, tools: dict) -> ProofPlan: ...

# ===== Solver =====
class Solution(BaseModel):
    answer: str
    proof: List[Step]
    confidence: float
    verified: bool

def solve(plan: ProofPlan, graph: GeometryGraph) -> Solution: ...

```

2.1 接口契约原则

- **类型安全**: Pydantic 校验输入输出。
- **版本化**: 每个数据结构带 version 字段。
- **幂等**: 纯函数，相同输入相同输出（便于缓存/重放）。
- **可降级**: 失败返回错误码 + 原因，不抛裸异常。

3. 项目结构

```
geometry_agent/
├── design/                # 设计文档
├── src/geometry_agent/
│   ├── perception/       # Geometry Parser
│   │   ├── preprocess.py
│   │   ├── detectors/    # point/line/circle/ellipse/mark
│   │   ├── fitting.py
│   │   └── orchestrator.py
│   ├── graph/           # Geometry Graph
│   │   ├── schema.py
│   │   ├── builder.py
│   │   └── queries.py
│   ├── agents/          # Relation Extraction Agents
│   │   ├── base.py
│   │   ├── point_agent.py
│   │   ├── line_agent.py
│   │   ├── circle_agent.py
│   │   ├── ellipse_agent.py
│   │   ├── polygon_agent.py
│   │   ├── mark_agent.py
│   │   ├── cross_agent.py
│   │   └── scheduler.py
│   ├── verifier/        # Constraint Verifier
│   │   ├── verifiers/   # 各关系验证器
│   │   ├── tolerance.py
│   │   └── engine.py
│   ├── dsl/             # DSL parser/serializer
│   │   ├── grammar.lark
│   │   ├── parser.py
│   │   └── serializer.py
│   ├── reasoning/       # LLM Reasoning Agent
│   │   ├── prompts/
│   │   ├── agent.py
│   │   ├── tot.py
│   │   ├── reflection.py
│   │   └── voting.py
│   ├── solver/          # Symbolic Solver
│   │   ├── sympy_engine.py
│   │   ├── z3_engine.py
│   │   ├── rule_engine.py
│   │   └── lean_bridge.py
│   ├── data/            # 数据集与合成
│   │   ├── synth/
│   │   └── augment.py
```

```

|   |   └── loader.py
|   ├── theorems/          # 定理库 + RAG
|   ├── api/               # FastAPI
|   |   └── routes.py
|   ├── config.py
|   └── pipeline.py        # 端到端编排
└── tests/
    ├── unit/
    ├── regression/
    └── e2e/
└── configs/
└── prompts/fewshot/
└── pyproject.toml

```

4. 端到端编排 (Pipeline)

```

class GeometryPipeline:
    def __init__(self, cfg):
        self.parser = GeometryParser(cfg.parser)
        self.graph_builder = GraphBuilder()
        self.agent_scheduler = AgentScheduler(cfg.agents)
        self.verifier = VerifierEngine(cfg.verifier)
        self.dsl_ser = DSLSerializer()
        self.llm_agent = LLMReasoningAgent(cfg.llm, tools=self._tools())
        self.solver = SymbolicSolver(cfg.solver)

    def _tools(self):
        return {"verify": self.verifier.verify,
                "solve": self.solver.solve,
                "search": self.theorems.search,
                "graph_query": self.graph.query}

    def run(self, image, text):
        prims = self.parser.parse(image, text)
        graph = self.graph_builder.build(prims)
        candidates = self.agent_scheduler.extract(graph)
        graph = self.verifier.verify(candidates, graph)
        dsl = self.dsl_ser.to_dsl(graph)
        plan = self.llm_agent.reason(dsl, text, self._tools())
        solution = self.solver.solve(plan, graph)
        return self._assemble(solution, graph)

```

5. 部署方案

5.1 单机部署 (Docker Compose)

```
services:
  api:      # FastAPI 主服务
  worker:   # 异步解题 worker (Celery)
  sam:      # SAM 推理服务 (GPU)
  yolo:     # YOLO 推理服务 (GPU/CPU)
  vector:   # FAISS/Chroma 定理库
  redis:    # 任务队列 + 缓存
```

5.2 模型部署

- **本地 GPU**: SAM、YOLO 本地推理 (单卡足够)。
- **LLM API**: GPT-4o/Claude/Qwen 走商用 API。
- **回退**: LLM API 失败回退本地 Qwen2.5。

5.3 扩展性

- 模块无状态, 可水平扩展。
 - 重计算 (感知、关系抽取) 用进程池/ worker 扩展。
 - LLM 调用走 API, 无需扩展。
-

6. 可观测性

6.1 结构化日志

每个模块输出 JSON Lines 日志:

```
{"ts": "...", "module": "parser", "step": "deskew", "input_hash": "...", "duration_ms": 12, "result": {...}}
```

6.2 verification_log

全程累积验证日志, 随最终结果输出, 支持回放调试。

6.3 指标监控

- 各模块延迟、成功率。
- LLM token 消耗、工具调用次数。
- 端到端准确率 (评估集定期跑)。

6.4 回放

保存每题的中间产物 (PrimitiveSet/Graph/DSL/ProofPlan/Solution), 支持单题回放调试。

7. 测试策略

层次	方法	工具
单元测试	各函数/验证器	pytest
模块测试	模块输入输出契约	pytest + Pydantic
回归测试	合成数据（带 GT）	合成回归集
端到端测试	真题评估集	评估脚本
一致性测试	DSL<=>Graph 往返	随机图生成
性能测试	延迟/内存	pytest-benchmark

7.1 回归集

对每类检测器/验证器/Agent 建立合成回归集（带 GT），每次迭代跑，防回退。CI 集成。

7.2 评估集

真题 hold-out 评估集，定期跑端到端准确率，跟踪系统进步。

8. 配置管理

所有阈值、模型路径、LLM 参数集中于 configs/*.yaml，Pydantic Settings 校验：

```
class ParserConfig(BaseModel):
    deskew_threshold_deg: float = 1.0
    skeleton_method: str = "zhang_suen"

class VerifierConfig(BaseModel):
    on_line_abs_tol: float = 2.0
    on_line_rel_tol: float = 0.005
    perp_angle_tol: float = 3.0

class LLMConfig(BaseModel):
    model: str = "gpt-4o"
    temperature: float = 0.3
    max_tool_calls: int = 30
    voting_n: int = 5
```

9. 设计路线小结

工程实现的设计路线为：“Python 技术栈 + 开源模型优先 → Pydantic 接口契约 → 模块化项目结构 → Pipeline 编排 → Docker Compose 部署 → 结构化日志 + verification_log 可观测 → 合成回归 + 真题评估双测试 → YAML 配置集中管理”。强调类型安全、可观测、可测试、可扩展，为系统迭代提供可靠工程基座。